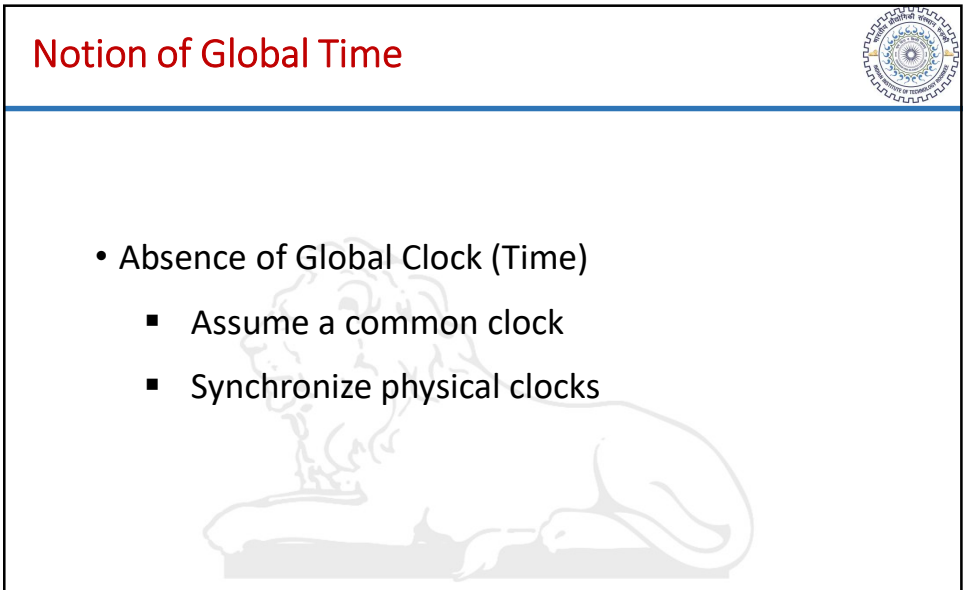
Distributed Systems

Design Issues
Issue 1: Time in Distributed Systems

Distributed system

- **Inherent limitations of distributed systems**
  - ❖ Lack of common memory
  - ❖ Absence of a global (system wide) clock
  - ❖ Unpredictable communication delays
- **Difficulty in designing Solution for**
  - ❖ Ordering events in the absence of a global clock
  - ❖ Capturing the global state
  - ❖ Clock synchronization

Notion of Global Time



- Absence of Global Clock (Time)
  - Assume a common clock
  - Synchronize physical clocks

Limitation of Lamport's Clock

a → b implies C(a) < C(b)

BUT

C(a) < C(b) doesn't imply a → b !!

So not a true clock !!

Solution: Vector Clocks

C_i is a vector of size n (no. of processes) at process *i*

$C_i(a)$ is a vector of size n associated with event a referred as its timestamp

$C_i[i]$ – i's own logical time

$C_i[j]$ – i's best guess of the logical time at j ($j \neq i$)

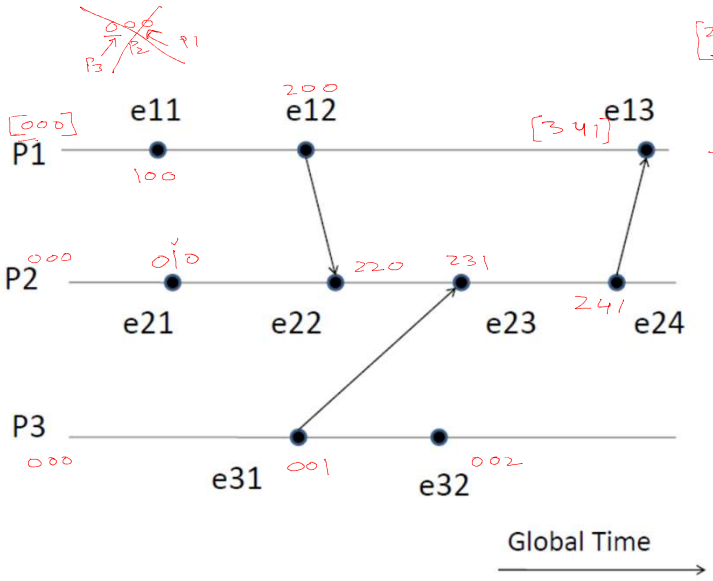
Update rules:

- $C_i[i] = C_i[i] + d$ for every event at process *i* ($d > 0$)
- if a is send of message m from *i* to j with vector timestamp $t_m = C_i(a)$, on receive of m at j:

  $C_j[k] = \max(C_j[k], t_m[k])$ for all k

Vector Clocks

- **Assertion.**
  - At any instant
  - for all i, j: $C_i[i] \geq C_j[i]$



For events a and b with vector timestamps t_a and t_b,

- $t_a = t_b$ iff for all i, $t_a[i] = t_b[i]$ — 1 2 3 = 1 2 3
- $t_a \neq t_b$ iff for some i, $t_a[i] \neq t_b[i]$ — 1 3 3 = 1 2 3
- $t_a \leq t_b$ iff for all i, $t_a[i] \leq t_b[i]$ — 1 3 3 >= 1 2 3
- $t_a < t_b$ iff ($t_a \leq t_b$ and $t_a \neq t_b$) — 1 3 3 > 1 2 3
- $t_a \,||\, t_b$ iff ($t_a \not\leq t_b$ and $t_b \not\leq t_a$)

Relation || is not a partial order because it is not transitive

A = (3, 4, 5), B = (2, 6, 3) and C = (3, 5, 5)

Q1 relationship between t_A and t_B?
Ans. t_A || t_B OR A || B

Q2 relationship between t_B and t_C?
Ans. B || C

Q2 relationship between t_A and t_C?
Ans. A < C

- a → b iff $t_a < t_b$
- Events a and b are causally related iff $t_a < t_b$ or $t_b < t_a$, else they are concurrent
- System of vector clocks allows us to order events and decide whether two events are causally related or not by simply looking at the timestamps of the events
- Total order?

  This is still not a total order

Voldemort is a distributed data store that was designed and used by **LinkedIn** for highly-scalable storage

Amazon **DynamoDB**

Hadoop Distributed File System

Google File System

Application of vector clocks: Causal ordering of messages:

- If send(m1)→ send(m2), then every recipient of both message m1 and m2 must "deliver" m1 before m2.
  - "deliver" – when the message is actually given to the application for processing
- Causal ordering of messages is needed for applications such as **replicated database systems**

Importance of ordering of messages

- A Replicated database at site1 and site2
- Initial balance in the account X is 1500
- Message1 (User1): Credit Rs 1000 in account X
- Message2 (User1): Debit Rs. 2000 from account X

After message 1 X will have 2500 (if message 1 is delivered first)